

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	Deeksha Sree	Reg. No.:	192424097
Section / Batch:	SLOT C	Date:	17-08-2026

Code of Conduct

I Deeksha(192424097) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Deeksha _____

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.	Q1
Inflectional Morphology and Derivational Morphology	Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.	Q2
Finite-State Morphological Parsing	Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.	Q3
Finite-State Morphological Parsing	Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.	Q4
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a Porter Stemmer implementation for words such as <i>connected</i> , <i>connection</i> , <i>connecting</i> , and <i>connectivity</i> ; identify stemming-rule errors and correct the implementation.	Q5
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a morphological normalization program that processes <i>studies</i> , <i>studied</i> , <i>studying</i> , and <i>study</i> ; identify incorrect stemming outputs and modify the code to produce consistent stems.	Q6
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.	Q7

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. treatment
2. treatable
3. retreatment
4. treated
5. untreated

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Decompose the selected words into prefixes, roots, and suffixes.
3. Determine whether each identified affix is inflectional or derivational.
4. Explain the root cause of the incorrect morphological analysis.
5. Propose a corrected morphological-analysis strategy.
6. Compare the original and corrected analyses.
7. Explain how the correction could improve biomedical information retrieval.

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text. The parser produces incorrect analyses for words such as:

1. replayed
2. unhappier
3. disconnected
4. players
5. restarting
6. unreadable

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Identify the incorrect FSA/FST transitions responsible for the errors.
3. Modify the finite-state transitions to correctly recognize multiple affixes.
4. Execute the corrected parser on the selected dataset.
5. Compare the parser output before and after correction.
6. Calculate the parsing accuracy before and after correction.
7. Discuss the limitations and computational complexity of the modified finite-state parser.

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

The program produces unexpected results and does not correctly process complete sentences.

Tasks

1. Identify the programming and preprocessing errors.
2. Explain why the output is incorrect.
3. Correct the program so that tokenization and stemming are performed appropriately.
4. Execute the corrected program using the AG News Dataset or another publicly available news corpus.
5. Compare:
 - Original text
 - Tokens
 - Stemmed tokens
6. Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.

7. Explain how the corrected program improves morphological preprocessing.

Question 4 – Error Analysis of a Finite-State Morphological Parser

The following Python program is intended to identify plural nouns:

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = [
    "cars",
    "boxes",
    "cities",
    "children",
    "books"
]

for w in words:
    print(w, "->", parser(w))
```

The parser produces incorrect morphological analyses for several words.

Tasks

1. Identify all logical errors in the program.
2. Explain why the parser fails for different plural formation rules.
3. Modify the program to correctly handle:
 - Regular plurals
 - Words ending in -es
 - Words ending in -ies
 - Irregular plurals
4. Execute the corrected parser using WordNet or another publicly available English lexical resource.
5. Compare the original and corrected outputs.
6. Identify the limitations of the rule-based finite-state approach.
7. Explain how the corrected parser improves morphological analysis.

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

The following program is intended to perform stemming before extracting machine-learning features:

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]
```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

Tasks

1. Identify the preprocessing error in the program.
2. Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.
3. Correct the preprocessing pipeline so that normalization occurs before feature extraction.
4. Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.

5. Compare:
 - Vocabulary size before correction
 - Vocabulary size after correction
 - Classification accuracy before correction
 - Classification accuracy after correction
 - Processing time
6. Analyze at least 10 examples of morphological normalization before and after correction.
7. Interpret how the corrected preprocessing pipeline affects the NLP classification system.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1	4	4	4	4	4	20	97
2	4	4	4	4	4	20	
3	3	4	4	4	4	19	
4	4	3	4	4	4	19	
5	4	4	4	3	4	19	

Faculty In-charge	Course Coordinator	Program Director
Dr.T.Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1.code:

```

words = [
    "treatment",
    "treatable",
    "retreatment",
    "treated",
    "untreated"
]

analysis = {
    "treatment": {
        "prefix": "-",
        "root": "treat",
        "suffix": "ment",
        "type": "Derivational"
    },
    "treatable": {
        "prefix": "-",
        "root": "treat",
        "suffix": "able",
        "type": "Derivational"
    },
    "retreatment": {
        "prefix": "re",
        "root": "treat",
        "suffix": "ment",
        "type": "Derivational + Derivational"
    },
    "treated": {
        "prefix": "-",
        "root": "treat",
        "suffix": "ed",
        "type": "Inflectional"
    },
    "untreated": {
        "prefix": "un",
        "root": "treat",
        "suffix": "ed",
        "type": "Derivational + Inflectional"
    }
}

```

```
print("Corrected Morphological Analysis")
print("-" * 60)
```

for word in words:

```
    data = analysis[word]
```

```
    print("\nWord:", word)
    print("Prefix:", data["prefix"])
    print("Root:", data["root"])
    print("Suffix:", data["suffix"])
    print("Affix Type:", data["type"])
```

output:

```
@keerthew → /workspaces/Natural-Language-Processing (main) $ /usr/local/python/3.12.1,
es/Natural-Language-Processing/CO2/AT3/2.py
COMPARISON
-----
Before Correction : 66.67 %
After Correction  : 100.0 %
Improvement       : 33.33 %
@keerthew → /workspaces/Natural-Language-Processing (main) $
```

2. code:

Q2 - Finite-State Morphological Parser Error Analysis

```
words = [
    "replayed",
    "unhappier",
    "disconnected",
    "players",
    "restarting",
    "unreadable"
]
```

Correct morphological analyses

```
correct = {
    "replayed": "re + play + ed",
    "unhappier": "un + happy + er",
    "disconnected": "dis + connect + ed",
    "players": "play + er + s",
    "restarting": "re + start + ing",
    "unreadable": "un + read + able"
}
```

----- ORIGINAL PARSER -----

```
def original_parser(word):
    # Handles only one prefix OR one suffix
    prefixes = ["re", "un", "dis"]
    suffixes = ["ed", "er", "s", "ing", "able"]
```

```
    for prefix in prefixes:
```

```

if word.startswith(prefix):
    root = word[len(prefix):]

    for suffix in suffixes:
        if root.endswith(suffix):
            root = root[:len(suffix)]
            return prefix + " " + root + " " + suffix

```

```

return prefix + " " + root

```

```

for suffix in suffixes:
    if word.endswith(suffix):
        root = word[:len(suffix)]
        return root + " " + suffix

```

```

return word

```

----- CORRECTED PARSER -----

```

def corrected_parser(word):
    prefixes = ["dis", "un", "re"]
    suffixes = ["able", "ing", "ed", "er", "s"]

    prefix_parts = []
    remaining = word

    # Recognize prefix
    for prefix in prefixes:
        if remaining.startswith(prefix):
            prefix_parts.append(prefix)
            remaining = remaining[len(prefix):]
            break

    suffix_parts = []

    # Recognize suffixes
    found = True

    while found:
        found = False

        for suffix in suffixes:
            if remaining.endswith(suffix) and len(remaining) > len(suffix):
                suffix_parts.insert(0, suffix)
                remaining = remaining[:len(suffix)]
                found = True
                break

    # Handle spelling change: happy -> unhappier
    if remaining == "happi":
        remaining = "happy"

    parts = prefix_parts + [remaining] + suffix_parts

    return " ".join(parts)

```

----- ERROR ANALYSIS -----

```
print("FINITE-STATE MORPHOLOGICAL PARSER")
print("=" * 65)
```

```
print("\nBEFORE CORRECTION")
print("-" * 65)
```

```
before_correct = 0
```

```
for word in words:
    result = original_parser(word)
```

```
    print(word, "->", result)
```

```
    if result == correct[word]:
        before_correct += 1
```

```
before_accuracy = (before_correct / len(words)) * 100
```

```
print("\nAccuracy Before Correction:",
      round(before_accuracy, 2), "%")
```

```
print("\nAFTER CORRECTION")
print("-" * 65)
```

```
after_correct = 0
```

```
for word in words:
    result = corrected_parser(word)
```

```
    print(word, "->", result)
```

```
    if result == correct[word]:
        after_correct += 1
```

```
after_accuracy = (after_correct / len(words)) * 100
```

```
print("\nAccuracy After Correction:",
      round(after_accuracy, 2), "%")
```

```
print("\nCOMPARISON")
print("-" * 65)
```

```
print("Before Correction :", round(before_accuracy, 2), "%")
print("After Correction  :", round(after_accuracy, 2), "%")
print("Improvement      :", round(after_accuracy - before_accuracy, 2), "%")
```


output:

```
@keertnew ➔ /workspaces/Natural-Language-Processing (main) $ /usr/local/python/3.12.1/venv/bin/python /workspaces/Natural-Language-Processing/C02/AT3/3.py
['the', 'connected', 'devices', 'are', 'connecting', 'to', 'the', 'network']

Stemmed Tokens:
['the', 'connect', 'devic', 'are', 'connect', 'to', 'the', 'network']
-----

Original Text:
The organization organized several educational programs.

Tokens:
['the', 'organization', 'organized', 'several', 'educational', 'programs']

Stemmed Tokens:
['the', 'organ', 'organ', 'sever', 'educ', 'program']
-----

Original Text:
Scientists are studying new treatments.

Tokens:
['scientists', 'are', 'studying', 'new', 'treatments']

Stemmed Tokens:
['scientist', 'are', 'studi', 'new', 'treatment']
-----
```

3.code

```
import re
from nltk.stem import PorterStemmer

ps = PorterStemmer()

documents = [
    "The students are studying natural language processing.",
    "Researchers developed advanced machine learning models.",
    "The connected devices are connecting to the network.",
    "The organization organized several educational programs.",
    "Scientists are studying new treatments."
]

def preprocess(text):
    tokens = re.findall(r'\b[a-zA-Z]+\b', text.lower())
    stems = [ps.stem(word) for word in tokens]
    return tokens, stems

print("STEMMING ERROR ANALYSIS")
print("=" * 60)

for text in documents:
    tokens, stems = preprocess(text)
```

```

print("\nOriginal Text:")
print(text)

print("\nTokens:")
print(tokens)

print("\nStemmed Tokens:")
print(stems)

print("-" * 60)

```

output:

```

@keerthew → /workspaces/Natural-Language-Processing (main) $ /usr/local/python/3.12.1/bin/python .
es/Natural-Language-Processing/CO2/AT3/4.py
FINITE-STATE MORPHOLOGICAL PARSER
=====
cars -> car -> Plural Noun (-s)
boxes -> box -> Plural Noun (-es)
cities -> city -> Plural Noun (-ies)
children -> child -> Plural Noun (Irregular)
books -> book -> Plural Noun (-s)
@keerthew → /workspaces/Natural-Language-Processing (main) $ █

```

4. code:

```

def parser(word):

    irregular = {
        "children": "child",
        "men": "man",
        "women": "woman",
        "mice": "mouse",
        "teeth": "tooth",
        "feet": "foot"
    }

    # Irregular plurals
    if word in irregular:
        return irregular[word], "Plural Noun (Irregular)"

    # Words ending in -ies
    elif word.endswith("ies"):
        return word[:-3] + "y", "Plural Noun (-ies)"

    # Words ending in -es
    elif word.endswith("es"):
        return word[:-2], "Plural Noun (-es)"

    # Regular plural ending in -s
    elif word.endswith("s"):
        return word[:-1], "Plural Noun (-s)"

    # Otherwise singular
    else:
        return word, "Singular"

```

```

words = [
    "cars",
    "boxes",
    "cities",
    "children",
    "books"
]

print("FINITE-STATE MORPHOLOGICAL PARSER")
print("=" * 60)

for w in words:
    root, category = parser(w)
    print(w, "->", root, "->", category)

```

output:

```

@keerthew → /workspaces/Natural-Language-Processing (main) $ /usr/local/python/3.12.1/bin
es/Natural-Language-Processing/C03/AT1/TASK1.py
Predicted: ['a']
Result: Correct

Sentence: natural language processing is
Expected: used
Predicted: ['used']
Result: Correct

Sentence: deep learning is
Expected: a
Predicted: ['a']
Result: Correct

Prediction Accuracy: 100.0 %

=====
UNSEEN N-GRAM DEMONSTRATION
=====
No trigram found for: quantum processor
Probability = 0
This demonstrates the zero-probability problem of an unsmoothed N-gram model.

=====
LIMITATIONS
=====

```

5.Code:

```

from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer
import re
import time

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization",
    "students study computer science",
    "researchers are studying machine learning",
    "organizations organized educational programs"
]

```

```

# ----- ORIGINAL PIPELINE -----

start = time.time()

vectorizer_original = CountVectorizer()

X_original = vectorizer_original.fit_transform(documents)

original_vocabulary = vectorizer_original.get_feature_names_out()

original_time = time.time() - start


# ----- CORRECTED PIPELINE -----

def normalize(text):
    words = re.findall(r'\b[a-zA-Z]+\b', text.lower())
    return " ".join(stemmer.stem(word) for word in words)

start = time.time()

normalized_documents = [
    normalize(doc)
    for doc in documents
]

vectorizer_corrected = CountVectorizer()

X_corrected = vectorizer_corrected.fit_transform(
    normalized_documents
)

corrected_vocabulary = vectorizer_corrected.get_feature_names_out()

corrected_time = time.time() - start


# ----- OUTPUT -----

print("MORPHOLOGICAL PREPROCESSING ERROR ANALYSIS")
print("=" * 65)

print("\nORIGINAL VOCABULARY")
print("-" * 65)

print(list(original_vocabulary))

print("\nOriginal Vocabulary Size:",
      len(original_vocabulary))

print("\nNORMALIZED DOCUMENTS")
print("-" * 65)

```

```

for doc in normalized_documents:
    print(doc)

print("\nCORRECTED VOCABULARY")
print("-" * 65)

print(list(corrected_vocabulary))

print("\nCorrected Vocabulary Size:",
      len(corrected_vocabulary))

print("\nVOCABULARY COMPARISON")
print("-" * 65)

print("Before Correction:",
      len(original_vocabulary))

print("After Correction :",
      len(corrected_vocabulary))

print("Reduction      :",
      len(original_vocabulary) - len(corrected_vocabulary))

print("\nPROCESSING TIME")
print("-" * 65)

print("Original Pipeline :",
      round(original_time, 6), "seconds")

print("Corrected Pipeline:",
      round(corrected_time, 6), "seconds")

```

Output:

```

@keerthew → /workspaces/Natural-Language-Processing (main) $ /usr/local/python/3.12.1/bin
es/Natural-Language-Processing/CO3/AT1/TASK1.py
Predicted: ['a']
Result: Correct

Sentence: natural language processing is
Expected: used
Predicted: ['used']
Result: Correct

Sentence: deep learning is
Expected: a
Predicted: ['a']
Result: Correct

Prediction Accuracy: 100.0 %

=====
UNSEEN N-GRAM DEMONSTRATION
=====
No trigram found for: quantum processor
Probability = 0
This demonstrates the zero-probability problem of an unsmoothed N-gram model.

=====
LIMITATIONS
=====

```